

Quarter 4: Project Proposal

Nerdle: Math-based Formula Guessing Game

Problem Statement:

Many students find it difficult to understand, remember, and correctly apply mathematical formulas, especially when learning them the traditional way. Because of this, students lose interest, leading to low engagement, limited critical thinking, and difficulty in solving problems that require proper formula usage. There are not many websites and apps that provide user-friendly tools that make learning mathematical formulas both fun and interactive. Moreover, students have different skill levels, and not all activities are able to provide different difficulties to match their abilities. Therefore, there is a need to develop a game-based application that combines fun and learning, helping users practice formulas, improve their skills, and learn in a more enjoyable way.

Project Objectives:

- To develop a wordle-inspired math game that makes you guess the correct use of the given formula.
- To familiarize the user with the given formulas and their use.
- To enhance critical thinking and problem-solving skills.
- To implement different difficulties, depending on the user's skill level.
- To make a user-friendly interface that mixes fun with learning.

Planned Features:

1. Countless Attempts
 - Players may have endless attempts to get the correct formula
2. Color Hint System
 - Highlighted letters; green, yellow, and red
3. Streak System
 - The number of games the player has played
4. Difficulty Levels
 - Easy: Basic formulas, Medium: multi-variable formulas, Hard: Complex formulas
5. Formula Guessing Mode
 - Players have the option to have the formula display hidden or shown
6. Leaderboard
 - Shows the players stats based on previous attempts

Planned Inputs and Outputs:

Inputs:

- Player Name
- Menu Selection (Formula-Based Dictionary/Difficulty Choices/Player's Stats)
- Formula Display (hidden/shown)
- Player Guesses
- Play Again Option (Yes/No)

Outputs:

- Welcome Message and Game Introduction
- Menu Options Display
- Formula-Based Dictionary (When chosen, the program displays the list of all formulas that will be used in the game)
- Player's Stats
 - Number of wins
 - Number of Losses
 - Number of Games Played
 - Win Rate
- Formula Display Option
- Number of Letters the Formula Has
- The Number of Guesses the Player Has
- The Highlighted Letters Based on the Placement of the Letters of the Player's Guess
 - Green = correct letter and position
 - Yellow = correct letter, wrong position
 - Red = not in the formula
- Win Message "You have guessed the correct answer"
- Lose Message "You have ran out of guesses"
- Play Again Prompt
- Exit/Game Termination

Logic Plan:

START

Asks the player for his/her name

Welcomes the player with a short introduction about the game

Asks the player to select an option shown in the menu selection repeatedly, until he/she exits

If the player chooses the Formula-based dictionary:

The program shows the list of formulas that may appear in the game

The program asks the player to select an option again

If the player chooses to see his/her stats:

The program displays the player's stats (Number of Wins, Number of Losses, Number of Games played, Winrate)

The program asks the player to select an option again

If the player chooses to start the game:

The program asks the player to choose a difficulty

The program starts with showing a formula at the top of the screen (Ex. $b \cdot h$)

The program informs the player the number of letters the formula has (Ex. 15 Letters)

The program asks the player to enter his/her guess until he/she either guesses the right answer or guessed 10/7/5 times based on the selected difficulty (There is no limit on the number of characters, but his/her guess must be a formula in the list)

If the player's guess is incorrect:

The program highlights the letters in the right placement the color green, the program highlights the letters in the wrong placement but are in the word the color yellow, and the program highlights the letters that aren't in word the color red. (Ex.

AREA OF A RECTANGLE, Correct answer = AREA OF A TRIANGLE)

If the player's guess is correct:

The program displays "You have guessed the correct answer"

The program updates the player's stats

The program then asks the player if he/she wants to start guessing again

If the player answers yes:

The game will start again

If the player answers no:

The program asks the player to select an option again (Based on the menu selection)

If the player's guess is incorrect and guessed 10/7/5 times (based on the difficulty):

The program displays "You have ran out of guesses"

If the player answers yes:

The game will start again

If the player answers no:

The program asks the player to select an option again (Based on the menu selection)

If the player chooses to exit the game:

The game stops

Source Code:

```
import time
import random
import json
import os

STATS_FILE = 'stats.json'

def load_stats():
    if os.path.exists(STATS_FILE):
        with open(STATS_FILE, 'r') as f:
            return json.load(f)
    return {}

def save_stats(stats):
    with open(STATS_FILE, 'w') as f:
        json.dump(stats, f, indent=4)

def update_player_stats(name, won):
    stats = load_stats()
    if name not in stats:
        stats[name] = {"games_played": 0, "wins": 0}

    stats[name]["games_played"] += 1
    if won:
        stats[name]["wins"] += 1

    save_stats(stats)

def colors(formula, formula_guess):
    inputted_user = input("Enter your guess: ")
    inputted = list(inputted_user)

    green = '\033[42m'
    yellow = '\033[43m'
    red = '\033[41m'
    reset = '\033[0m'

    temp_guess = list(formula_guess)
    output = []
    res_list = [red] * len(inputted)

    for i in range(len(inputted)):
```

```

        if i < len(temp_guess) and inputted[i].lower() == temp_guess[i].lower():
            res_list[i] = green
            temp_guess[i] = "#"
    for i in range(len(inputted)):
        if res_list[i] == green: continue
        for j in range(len(temp_guess)):
            if inputted[i].lower() == temp_guess[j].lower():
                res_list[i] = yellow
                temp_guess[j] = "#"
                break

    for i in range(len(inputted)):
        print(f"{res_list[i]}{inputted[i]}{reset}", end="")
    print()

    return inputted_user

def start_game(name, Formula_Based_Dictionary, Topics):
    difficulty = 0
    while difficulty not in range(1, 5):
        try:
            difficulty = int(input(f"
What difficulty would you like to choose?
-----
1 - Easy (8 Attempts)
2 - Normal (6 Attempts)
3 - Hard (4 Attempts)
4 - Endless (Infinite Attempts)
-----
Selection: "))
        except ValueError:
            continue

    attempts_allowed = {1: 8, 2: 6, 3: 4, 4: 999}[difficulty]

    formula_pair = random.choice(Formula_Based_Dictionary[Topics-1])
    target_answer = formula_pair[1]

    print(f"""\nTarget: {formula_pair[0]}
Hint: The formula has {len(target_answer)} characters.\n""")

    inputted_user = ""
    current_attempt = 0
    won = False

```

```

while current_attempt < attempts_allowed:
    inputted_user = colors(target_answer, list(target_answer))
    current_attempt += 1

    if inputted_user.lower() == target_answer.lower():
        print(f"Correct! You got it in {current_attempt} tries.\n")
        won = True
        break

    if difficulty != 4:
        print(f"Attempts left: {attempts_allowed - current_attempt}")

if won != True:
    print(f"\nGame Over! The correct answer was: {target_answer}")

update_player_stats(name, won)

name = input("What should we call you? ").strip()
stats_data = load_stats()

Formula_Based_Dictionary = [
    [['Area of a Square', 's^2'], ['Area of a Rectangle', 'w*h'], ['Area of a Triangle', 'b*h/2'], ['Rhombus',
    '(D*d)/2'], ['Trapezoid', '(B*b)/2*h'], ['Regular polygon', '(P/2)*a'], ['Circle', 'πr^2']],
    [['Commutative Property', 'a*b=b*a'], ['Associative Property', 'a(b*c)=(a*b)c'], ['Distributive
    Property', 'a*(b+c)=a*b+a*c'], ['Identity Property', 'a*1=a'], ['Inverse Property', 'a*1/a=1']],
    [['Square', '4l'], ['Rectangle', '2(w+h)'], ['Triangle', 'a+b+c'], ['Circle', '2πr']],
    [['Square of a Binomial', 'a^2+2ab+b'], ['Difference of Squares', '(a-b)(a+b)'], ['Product of Sum and
    Differences', '(a-b)(a+b)']]
]

menu_selection = 0
while menu_selection != 4:
    print(f"\n--- NERDLE MENU (Player: {name}) ---")
    try:
        menu_selection = int(input("""1 - View Formulas\n2 - View My Stats\n3 - Start Game\n4 -
        Exit\nSelection: """))
    except ValueError:
        continue

    if menu_selection == 1:
        for category in Formula_Based_Dictionary:
            for item in category:
                print(f"{item[0]}: {item[1]}")

```

```

if menu_selection == 2:
    stats = load_stats()
    p = stats.get(name, {"games_played": 0, "wins": 0})
    if p['games_played'] > 0:
        wr = (p['wins'] / p['games_played'] * 100)
    else:
        wr = 0
    print(f"\n--- {name}'s Stats ---")
    print(f"Games Played: {p['games_played']}")
    print(f"Wins: {p['wins']}")
    print(f"Win Rate: {wr:.1f}%")

if menu_selection == 3:
    try:
        topic = int(input("\n1 - Area of 2D Shapes\n2 - Properties of Real Numbers\n3 - Perimeter
of 2D Shapes\n4 - Special Products\nSelection: "))
        start_game(name, Formula_Based_Dictionary, topic)
    except (ValueError, IndexError):
        print("Invalid topic selection.")

print("Thanks for playing!")

```

References:

Dyscalculia. (2025, October 13). Cleveland Clinic.

<https://my.clevelandclinic.org/health/diseases/23949-dyscalculia>

Hjelle, G. A. (2023, December 1). Build a Wordle clone with Python and Rich.

<https://realpython.com/python-wordle-clone/>